

LECTURE 5 – STATE ESTIMATION (I)

INTRODUCTION TO AUTONOMOUS SYSTEMS · MODULE III

Topics: Hidden-state problem · Belief · Bayes filter · Corridor example · Gaussian machinery (opening)

AGENDA

S	TOPIC	KEY IDEAS
1	The Hidden-State Problem	Belief · filtering vs prediction vs smoothing
2	The Bayes Filter	Predict / update · two Markov assumptions · corridor example
3	Gaussian Machinery (<i>opening</i>)	When can the filter be computed exactly?

§3 (cont.)–§7 — *Kalman filter, sensor fusion, particle filter, belief to policy* — are in Lecture 6.

LITERATURE

TOPIC	REFERENCE
Bayes, Gaussian and particle filters	Thrun, Burgard & Fox — <i>Probabilistic Robotics</i> (2005), Ch. 2–4
Rigorous filtering and smoothing	Särkkä — <i>Bayesian Filtering and Smoothing</i> (2013)
Kalman filter theory, Riccati, steady state	Anderson & Moore — <i>Optimal Filtering</i> (1979), Ch. 3–5
Original paper	Kalman — <i>A New Approach to Linear Filtering and Prediction Problems</i> (1960)

S1 · THE HIDDEN-STATE PROBLEM

THE AGENT NEEDS X_T BUT ONLY EVER SEES O_T



WHY STATE ESTIMATION IS NECESSARY

From now on the observation is **noisy**:

$$o_t = h(x_t) + v_t \qquad \text{or, more generally,} \qquad o_t \sim p(o_t \mid x_t).$$

PROBLEM	SOURCE	EXAMPLE	WHAT FIXES IT
Information loss	h not injective: $\dim \mathcal{O} < \dim \mathcal{X}$, or a degenerate Jacobian	GPS measures position, not velocity	Time: combine observations through f (observability, Lec 2)
Noise	v_t corrupts every reading	GPS error of a few metres	Averaging: combine with a prior, weighted by reliability

One estimator handles both: keep a **distribution over states**, propagate it through f , correct it with each o_t .

Time convention. $x_0 \rightarrow$ apply $a_0 \rightarrow x_1 \rightarrow$ observe $o_1 \rightarrow$ apply $a_1 \rightarrow \dots$

THE BELIEF

Definition (Belief). The **belief** at time t is the posterior over the current state given everything the agent has seen and done:

$$b_t(x) = p(x_t = x \mid o_{1:t}, a_{0:t-1}).$$

The **predicted belief** is the same quantity before o_t arrives:

$$\bar{b}_t(x) = p(x_t = x \mid o_{1:t-1}, a_{0:t-1}).$$

PROBLEM	TARGET	USES	TYPICAL USE
Filtering	$p(x_t \mid o_{1:t})$	past + present	online control — <i>this lecture</i>
Prediction	$p(x_{t+k} \mid o_{1:t}), k > 0$	past + present	planning, collision checking
Smoothing	$p(x_t \mid o_{1:T}), T > t$	past, present, future	offline analysis, mapping, training data

A controller can only use filtering and prediction: it cannot wait for the future.

SMOOTHING – AND IS TRAINING ON THE FUTURE CHEATING?

The name. A filtered trajectory is jagged and lags; a smoothed one is constrained by data on both sides. Formally, past and future are conditionally independent given x_t :

$$p(x_t \mid o_{1:T}) \propto p(o_{t+1:T} \mid x_t, o_{1:t}) p(x_t \mid o_{1:t}) = \underbrace{p(x_t \mid o_{1:t})}_{\text{filter}} \underbrace{p(o_{t+1:T} \mid x_t)}_{\text{evidence from the future}}$$

Leakage only if a deployed model receives hindsight as input.

- ✗ Policy trained on smoothed states, run on filtered ones → self-inflicted distribution shift (Lec 11).
- ✓ Hindsight as **labels** for a causal model: fitting $\hat{f}$ on smoothed (x_t, a_t, x_{t+1}) (Lec 15), training $o_t \mapsto x_t$.

S2 · THE BAYES FILTER

PREDICT THROUGH F , UPDATE WITH H



SETUP

- **Markov dynamics:** $x_t \sim p(\cdot \mid x_{t-1}, a_{t-1})$
- **Markov observation:** $o_t \sim p(\cdot \mid x_t)$
- **Actions from past observations only:** a_{t-1} is a (possibly random) function of $o_{1:t-1}$
- **Current belief:** $b_{t-1}(x_{t-1}) = p(x_{t-1} \mid o_{1:t-1}, a_{0:t-2})$

Action a_{t-1} is applied, observation o_t arrives. We want b_t .

Write $\mathcal{D}_{t-1} = (o_{1:t-1}, a_{0:t-1})$ for the data available just before o_t .

DERIVATION – STEPS 1-3

Step 1 – Bayes' rule on the newest observation:

$$p(x_t \mid o_t, \mathcal{D}_{t-1}) = \frac{p(o_t \mid x_t, \mathcal{D}_{t-1}) p(x_t \mid \mathcal{D}_{t-1})}{p(o_t \mid \mathcal{D}_{t-1})}$$

Step 2 – Markov observation. $o_t \perp \mathcal{D}_{t-1} \mid x_t$, so

$$p(o_t \mid x_t, \mathcal{D}_{t-1}) = p(o_t \mid x_t).$$

Step 3 – Marginalise over the previous state:

$$p(x_t \mid \mathcal{D}_{t-1}) = \int p(x_t \mid x_{t-1}, \mathcal{D}_{t-1}) p(x_{t-1} \mid \mathcal{D}_{t-1}) dx_{t-1}$$

DERIVATION – STEPS 4-6

Step 4 – Markov dynamics. $x_t \perp (o_{1:t-1}, a_{0:t-2}) \mid (x_{t-1}, a_{t-1})$, so

$$p(x_t \mid x_{t-1}, \mathcal{D}_{t-1}) = p(x_t \mid x_{t-1}, a_{t-1}).$$

Step 5 – Actions carry no extra information. a_{t-1} was computed from $o_{1:t-1}$, so it says nothing about x_{t-1} that $o_{1:t-1}$ does not:

$$p(x_{t-1} \mid \mathcal{D}_{t-1}) = p(x_{t-1} \mid o_{1:t-1}, a_{0:t-2}) = b_{t-1}(x_{t-1}).$$

Step 6 – Combine. The denominator of Step 1 does not depend on x_t : it is a normalising constant η .

THE BAYES FILTER

Theorem (Bayes filter). Under the Markov dynamics and Markov observation assumptions:

$$\textbf{Predict: } \bar{b}_t(x_t) = \int p(x_t \mid x_{t-1}, a_{t-1}) b_{t-1}(x_{t-1}) dx_{t-1}$$

$$\textbf{Update: } b_t(x_t) = \eta p(o_t \mid x_t) \bar{b}_t(x_t), \quad \eta^{-1} = \int p(o_t \mid x) \bar{b}_t(x) dx$$

Predict spreads. Push the belief through the dynamics; process noise adds uncertainty.

Update sharpens. Reweight each state by how well it explains the observation.

Estimation alternates between these two stages.

EXACTLY TWO ASSUMPTIONS

ASSUMPTION	FORMAL STATEMENT	WHERE IT COMES FROM
Markov observation	$o_t \perp (o_{1:t-1}, a_{0:t-1}) \mid x_t$	h depends only on the current state; sensor noise independent over time
Markov dynamics	$x_t \perp (o_{1:t-1}, a_{0:t-2}) \mid x_{t-1}, a_{t-1}$	The definition of state: x_t summarises everything relevant for the future

If either fails, the filter is wrong — **and more data will not fix it.**

A filter that drifts often means the state is missing something:

- unmodelled sensor bias · time-correlated disturbance (wind) · missing velocity

Remedy — state augmentation: add the missing quantity to x_t so that the Markov property holds again.

A RECIPE, NOT AN ALGORITHM

The recursion needs an **integral over $\mathcal{X}$** and a **product of functions on $\mathcal{X}$** . For continuous $\mathcal{X}$ there is no general way to store b_t or compute that integral.

Every practical filter is a choice of **representation** for b_t :

REPRESENTATION OF b_t	FILTER	EXACT WHEN
Probability vector over a finite grid	Histogram filter (§2.2)	$\mathcal{X}$ finite
Gaussian (μ_t, Σ_t)	Kalman filter (Lec 6, §4)	f, h linear; noise Gaussian
Gaussian, after local approximation	EKF / UKF (Lec 6, §6.1)	never — approximate
Weighted samples $\{(x^{(i)}, w^{(i)})\}$	Particle filter (Lec 6, §6.2)	as $N \rightarrow \infty$

THE CORRIDOR – SETUP

A robot on a **circular corridor** of 10 cells, $x \in \{0, \dots, 9\}$. Doors at cells **1, 2, 6** on the side wall.

```
side wall:  W  [D] [D]  W  W  W  [D]  W  W  W
cells:      0   1   2   3   4   5   6   7   8   9   → back to 0
              move right →
```

- **Door sensor:** $p(\text{door} \mid \text{door cell}) = 0.8$, $p(\text{door} \mid \text{wall cell}) = 0.1$
- **Action** "move right": shifts by one cell (deterministic, for short arithmetic)
- **Prior:** uniform, $b_0(x) = 0.1$ — the filter does not know where the robot starts

The readings are the data. The robot's true position never enters the computation.

**THE SENSOR READS "DOOR". WHERE COULD THE ROBOT
BE?**



$t = 1 - \text{OBSERVE "DOOR"}$

Multiply by the likelihood and normalise:

- door cells: $0.1 \cdot 0.8 = 0.08$
- wall cells: $0.1 \cdot 0.1 = 0.01$
- total: $3(0.08) + 7(0.01) = 0.31$

$$b_1(x) = \begin{cases} 8/31 \approx 0.26 & x \in \{1, 2, 6\} \\ 1/31 \approx 0.03 & \text{otherwise} \end{cases}$$

The belief is **trimodal**: the sensor cannot tell the three doors apart.

**THE THREE DOORS LOOK ALIKE. WHAT CAN THE ROBOT
DO TO TELL THEM APART?**



$t = 2$ – MOVE RIGHT (PREDICT)

The action selects the transition model: $p(x \mid x', a_1 = \text{right}) = 1$ iff $x = x' + 1 \pmod{10}$. So

$$\bar{b}_2(x) = \sum_{x'} p(x \mid x', a_1 = \text{right}) b_1(x') = b_1(x - 1).$$

(With "left": $b_1(x + 1)$; with "stay": $b_1(x)$.)

Rows multiplied by 31 to avoid fractions — the update normalises anyway:

```
cell:           0  1  2  3  4  5  6  7  8  9
31 × b_1:       1  8  8  1  1  1  8  1  1  1
31 × pred_2:    1  1  8  8  1  1  1  8  1  1    (cell 9's mass wraps to cell 0)
```

Peaks move from $\{1, 2, 6\}$ to $\{2, 3, 7\}$.

THE SENSOR READS "DOOR" AGAIN. WHICH CELL WINS?



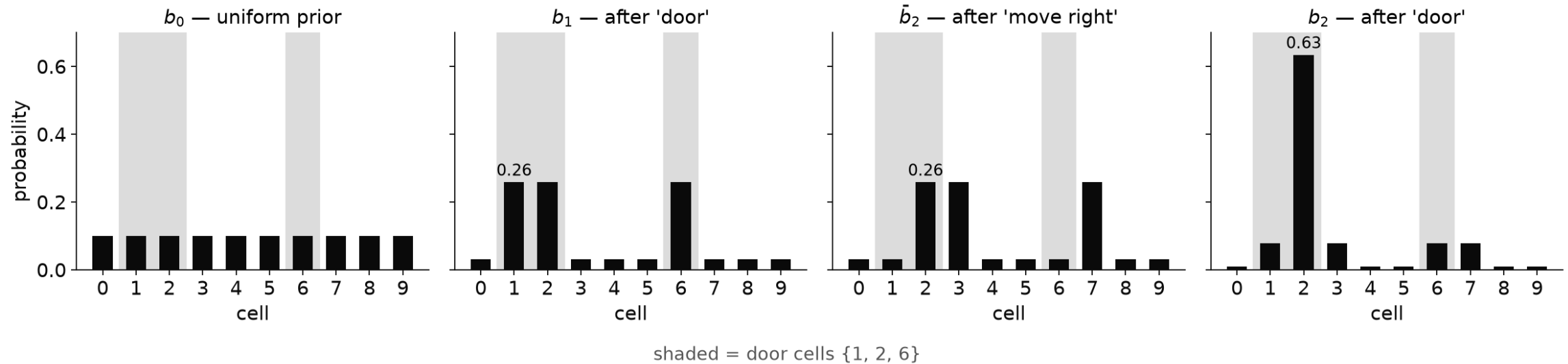
$t = 2$ – OBSERVE "DOOR" (UPDATE)

CELL	$31 \bar{b}_2$	DOOR?	$31 \bar{b}_2 \cdot p(o x)$
2	8	yes	6.4
3, 7	8	no	0.8 each
1, 6	1	yes	0.8 each
0, 4, 5, 8, 9	1	no	0.1 each

Column sum 10.1. Normalising:

$$b_2(2) \approx 0.63, \quad b_2(\{1, 3, 6, 7\}) \approx 0.08 \text{ each}, \quad b_2(\text{others}) \approx 0.01 \text{ each.}$$

THE BELIEF, STEP BY STEP



Most likely cell: $0.10 \rightarrow 0.26 \rightarrow 0.63$. The first "door" cuts ten hypotheses to three. The second step does the rest: only cell 1 has another door immediately to its right.

OTHER READING SEQUENCES

READINGS (o_1, o_2)	b_2 PEAKS AT	PEAK VALUE	PROBABILITY OF THIS SEQUENCE
door, door	cell 2	0.63	0.10
door, wall	cells 3, 7	0.34 each	0.21
wall, door	cells 1, 6	0.34 each	0.21
wall, wall	cells 0, 4, 5, 8, 9	0.17 each	0.48

- The last column follows from the map and models alone — computable offline.
- But it does not say where the robot is: **which row applies is decided by what the sensor reports.**
- The worked sequence is the lucky case. The most common outcome, "wall, wall", leaves five cells equally likely.

LESSONS FROM THE CORRIDOR (I)

Observability at work. Neither observation localises the robot alone; the two observations **plus the known motion** do. h alone is not injective, but (f, h) over several steps can be (Lec 2, §3).

Not located with certainty. $b_2(2) \approx 0.63$. "Door, door" could also come from cells 1, 3, 6, 7 with one false reading — exactly the cells with $b_2 \approx 0.08$. The peak only reaches 1 in the limit.

How does a robot that senses one cell score all ten? $b_t(x)$ is a set of **hypotheses**: "how plausible is it that I am at x ?" The robot carries a **map** (door positions), so it knows $p(o \mid x)$ for every x :

- hearing "door", it asks for each cell: *if I were there, how likely is this reading?*
- moving, it shifts every hypothesis by the same step.

Without a map, it must build one while localising — **SLAM**.

LESSONS FROM THE CORRIDOR (II)

Does it need to pass two doors? No. Read the map as a string:

```
W D D W W W D W W W      (cells 0-9)
```

- "door, door" occurs only at $1 \rightarrow 2$ · "wall, door, wall" only at $5 \rightarrow 6 \rightarrow 7 \rightarrow$ localised
- "door, wall" at $2 \rightarrow 3$ and $6 \rightarrow 7$ · "wall, wall, wall" at three places $\rightarrow$ ambiguous

Localisation happens once the **reading sequence becomes unique on the map**.

When it never works. Doors at cells 0 and 5 only: every sequence looks the same from x and $x + 5$. The belief stays **bimodal forever** — an unobservable system. Only a new sensor or a symmetry-breaking landmark helps.

Multimodality. Three modes at $t = 1$, two forever in the symmetric corridor. A single Gaussian cannot represent either — the first hint of the Kalman filter's limits (Lec 6, §6).

ON THE ROBOT

Models are prepared **offline** (likelihood vectors from the map, shift per action). The belief is computed **online**:

```
a = policy(b); robot.execute(a)
b = predict(b, a)                # e.g. np.roll(b, 1) for "right"
o = robot.read_sensor()          # one physical reading
b = b * (P_DOOR if o == "door" else P_WALL) # score all 10 hypotheses via the map
b = b / b.sum()
```

- Stores only the 10-number vector `b`, never the history (a lookup table needs 2^t rows).
- Policy options: act at `argmax(b)` (certainty equivalence) · commit once `b.max()` exceeds a threshold · choose actions that best separate surviving hypotheses (POMDP, Lec 12).

Cost. $O(|\mathcal{X}|^2)$ per step (general transitions), $O(|\mathcal{X}|)$ for local motion. A grid with k cells per dimension has k^n cells — fine for $n \leq 3$, hopeless beyond.

§3 · GAUSSIAN MACHINERY

PROVING THE CLOSURE PROPERTIES PROMISED IN LECTURE 4



**WHEN CAN THE PREDICT INTEGRAL AND THE UPDATE
PRODUCT BE COMPUTED EXACTLY?**



WHAT'S NEXT

Lecture 6 — State Estimation (II)

- §3 Gaussian machinery: the closure theorems and conjugacy that answer the question above
- §4 The Kalman filter — the Bayes filter for linear Gaussian models
- §5 Sensor fusion · §6 EKF / UKF and the particle filter
- §7 The belief as a sufficient statistic: estimator + controller